

SUPPLY MANAGEMENT SYSTEM

FSD Addendum - Section 26

Product Variants, Dynamic Attributes & Downstream Impact

Variant SKUs - Metadata-Driven Attributes - Dynamic Forms - Category Configuration - Full Module Impact Analysis

Version 1.0 | 2026

Document type	FSD Addendum - foundational product model evolution with full downstream impact
Impact scope	Products, Purchase Orders, GRN, Inventory, MIR, POS, Stock Ledger, Reports - every module that references product_id
Key decision	All transactional modules (PO, GRN, Inventory, MIR, POS) operate at VARIANT level, not product level. Product is the parent container; Variant is the sellable/purchasable/stockable unit.
Migration	DROP all existing product data and dependent transactional records (project in development). Clean rebuild with variant-first architecture.
New entities	ProductVariants, AttributeDefinitions, CategoryAttributes, VariantAttributeValues (all in inventory_schema)
Industry extensibility	Electronics, Garments, Furniture, Printing, Pharmacy, Automotive, Hardware - all supported without schema changes via the dynamic attribute engine
Version	1.0 2026

Table of Contents

- Table of Contents 2
- 1. Architecture Decision: Variant-First Model 3
 - 1.1 Why Variant-Level, Not Product-Level 3
 - 1.2 Products Without Variants 3
- 2. Revised Product Table 4
 - 2.1 Product - Revised Field Definitions 4
- 3. Product Variants 5
 - 3.1 ProductVariants - Field Definitions 5
 - 3.2 Worked Example: Dell Latitude 5450 5
 - 3.3 Worked Example: Cement (Simple Product, No Real Variants) 5
- 4. Dynamic Attributes 7
 - 4.1 AttributeDefinitions - Field Definitions 7
 - 4.2 CategoryAttributes - Linking Attributes to Categories 7
 - 4.3 VariantAttributeValues - Storing the Actual Values 8
 - 4.4 Worked Example: Laptop Category Attributes 8
- 5. Dynamic UI Rendering 9
 - 5.1 Rendering Flow 9
 - 5.2 Industry Examples - Same Engine, No Code Changes 9
- 6. Category Attribute Configuration (Admin) 10
- 7. Downstream Module Impact Analysis 11
 - 7.1 Purchase Orders (Recommendation: Option B - Purchase Variant) 11
 - 7.2 Goods Receipt (GRN) 11
 - 7.3 Inventory 11
 - 7.4 Material Issue Request (MIR) 11
 - 7.5 POS (if implemented) 12
 - 7.6 Reports, Supplier Scorecard, Cost Allocation 12
- 8. Search Strategy 13
 - 8.1 Search Implementation 13
- 9. Migration Strategy 14
 - 9.1 Migration Steps 14
 - 9.2 Rollback Strategy 14
- 10. API Endpoints 15
- 11. Performance and Indexing 16
- 12. Role-Based Access 17

1. Architecture Decision: Variant-First Model

The fundamental question across all 12 requirement items is: should the system transact at the Product level or the Variant level? This addendum makes the decision once, here, and every subsequent section implements it consistently.

Decision	ALL transactional modules operate at the VARIANT level. The Product table becomes a parent container (the 'Dell Latitude 5450'), and the ProductVariant table becomes the actual unit that is purchased, received, stocked, issued, sold, and tracked (the 'i7 / 16GB / 512GB' SKU). Every FK that currently points to product_id in PO lines, GRN lines, InventoryItems, MIR lines, and POS lines will point to variant_id instead.
----------	--

1.1 Why Variant-Level, Not Product-Level

- Different variants have different purchase prices (i5 costs less than i7) - PO lines must reference the specific variant being ordered.
- Different variants have different barcodes and SKUs - GRN receiving and POS scanning must resolve to a specific variant.
- Stock must be tracked per variant - knowing you have 50 'Dell Latitude 5450' is useless if you do not know how many are i5 vs i7.
- Serial/batch tracking is per variant - a serial number belongs to a specific SKU, not a generic product.
- This is the approach used by SAP (Material + Variant), Odoo (Product Template + Product Variant), and every modern ERP.

1.2 Products Without Variants

Backward compatibility	Products that do not need variants (e.g. Cement, Office Paper) still work: every Product automatically gets one default variant (is_default = true) with the same name, price, and barcode as the product. The system always transacts at variant level, but for simple products the user sees no difference - the default variant is auto-selected and the variant picker is hidden. This eliminates the need for a dual code path (product-level vs variant-level).
------------------------	---

2. Revised Product Table

The Product table is simplified to a parent/container entity. Price, barcode, and SKU fields move to ProductVariants. Fields that describe the product FAMILY remain on Product.

2.1 Product - Revised Field Definitions

Field	Type	Required	Notes
product_id	UUID	Yes	Primary key - unchanged
organization_id	UUID	Yes	Tenant discriminator (Addendum 25)
product_code	Text(50)	Yes (UQ per org)	e.g. DELL-LAT-5450. Identifies the product family, not a specific SKU.
product_name	Text(200)	Yes	Display name: 'Dell Latitude 5450'. Describes the family.
category_id	FK to Categories	Yes	Product category - drives which dynamic attributes appear (Section 4)
sub_category_id	FK to SubCategories	No	
description	Text(2000)	No	Rich text product description
brand	Text(100)	No	e.g. Dell, HP, Toyota. Shared across all variants.
unit	Lookup	Yes	Base UOM. Inherited by variants as default.
is_active	Boolean	Yes	
is_batch_tracked	Boolean	Yes	Applies to ALL variants of this product
is_serial_tracked	Boolean	Yes	Applies to ALL variants
is_direct_consumption	Boolean	Yes	Addendum 20 flag - applies to all variants
reorder_point	Decimal	No	Default for variants; overridable per variant
created at / created by	DateTime / FK	Yes	

REMOVED from Product (moved to ProductVariants): purchase_price, selling_price, barcode, last_purchase_price. These are variant-specific and no longer meaningful at the product family level.

3. Product Variants

3.1 ProductVariants - Field Definitions

Field	Type	Required	Notes
variant_id	UUID	Yes	Primary key. This is the ID referenced by PO lines, GRN lines, InventoryItems, MIR lines, POS, and every transactional entity.
product_id	FK to Products	Yes	Parent product family
organization_id	UUID	Yes	Tenant discriminator
sku	Text(50)	Yes (UQ per org)	Stock Keeping Unit - unique sellable/purchasable identifier. e.g. DELL-LAT-5450-I7-16-512
variant_name	Text(200)	Yes	Descriptive name: 'i7 / 16GB / 512GB SSD'. Auto-generated from attribute values or manually entered.
barcode	Text(50)	No (UQ per org if set)	EAN/UPC barcode for POS scanning and GRN receiving. Each variant can have a distinct barcode.
purchase_price	Decimal	Yes	Default purchase price for this specific variant
selling_price	Decimal	No	Retail/selling price for this variant (POS and sales)
last_purchase_price	Decimal	No (auto)	Updated on each GRN receipt for this variant
weight	Decimal	No	Weight in the product's base UOM weight unit (kg, lb)
dimensions	Text(100)	No	L x W x H as text, or structured JSON
is_default	Boolean	Yes	One variant per product must be is_default = true. For simple products (no real variants), this is the single auto-created variant.
is_active	Boolean	Yes	Default true
reorder_point	Decimal	No	Overrides Product.reorder_point for this specific variant if set
sort_order	Integer	No	Display ordering within the product's variant list
created_at / created_by	DateTime / FK	Yes	

3.2 Worked Example: Dell Latitude 5450

Default?	SKU	Variant Name	Purchase	Selling	Barcode
No	DELL-5450-I5-8-256	i5 / 8GB / 256GB	85,000	95,000	8901234567890
No	DELL-5450-I5-16-512	i5 / 16GB / 512GB	105,000	120,000	8901234567891
Yes	DELL-5450-I7-16-512	i7 / 16GB / 512GB	135,000	155,000	8901234567892

3.3 Worked Example: Cement (Simple Product, No Real Variants)

Default?	SKU	Variant Name	Purchase	Selling	Barcode
Yes	CEMENT-OPC-50KG	OPC 50kg Bag (default)	1,200	1,400	8901234500001

For Cement, the user never sees a variant picker. The system auto-selects the single default variant. All PO/GRN/Inventory/MIR lines reference this variant_id internally, but the UI shows only the product name.

4. Dynamic Attributes

Design principle	RAM, CPU, Color, Size, Material, IMEI are NOT columns in any table. They are metadata-driven attribute values stored in a generic key-value structure. This means adding a new attribute for a new industry (e.g. Thread Count for textiles, Dosage for pharmacy, Engine Displacement for automotive) requires zero schema changes - just an Admin creating a new AttributeDefinition.
-------------------------	--

4.1 AttributeDefinitions - Field Definitions

Field	Type	Required	Notes
<code>attribute_id</code>	UUID	Yes	Primary key
<code>organization_id</code>	UUID	Yes	Tenant-scoped. Each org defines its own attributes.
<code>attribute_name</code>	Text(50)	Yes (UQ per org)	Internal name: ram, cpu, color, material, dosage
<code>display_name</code>	Text(100)	Yes	User-visible label: RAM, CPU, Color, Material, Dosage
<code>data_type</code>	Lookup	Yes	TEXT / NUMBER / DECIMAL / DATE / BOOLEAN / DROPDOWN / MULTI_SELECT
<code>control_type</code>	Lookup	Yes	TEXTBOX / NUMBERBOX / DATEPICKER / TOGGLE / DROPDOWN / MULTI_SELECT / TEXTAREA
<code>dropdown_options</code>	NVARCHAR(MAX) JSON	Conditional	Required when <code>data_type</code> = DROPDOWN or MULTI_SELECT. JSON array of allowed values, e.g. ["Red","Blue","Green"] or [{"value":"4GB","label":"4 GB RAM"}]
<code>default_value</code>	Text(200)	No	Pre-filled value when creating a new variant
<code>validation_regex</code>	Text(200)	No	Optional regex for TEXT fields, e.g. IMEI pattern
<code>is_required</code>	Boolean	Yes	Default false. When true, every variant in a category with this attribute must have a value.
<code>is_searchable</code>	Boolean	Yes	Default false. When true, this attribute's values are included in the product search index (Section 8).
<code>is_filterable</code>	Boolean	Yes	Default false. When true, appears as a filter option on product list/POS screens.
<code>sort_order</code>	Integer	Yes	Display ordering on the form
<code>is_active</code>	Boolean	Yes	

4.2 CategoryAttributes - Linking Attributes to Categories

Field	Type	Required	Notes
<code>category_attribute_id</code>	UUID	Yes	Primary key
<code>category_id</code>	FK to Categories	Yes	Which category this attribute applies to
<code>attribute_id</code>	FK to AttributeDefinitions	Yes	Which attribute

Field	Type	Required	Notes
is_required	Boolean	Yes	Can override the attribute's default is_required for this specific category. e.g. Color is optional globally but required for Garments.
display_order	Integer	Yes	Order this attribute appears on the form for this category. Overrides the attribute's global sort_order.
organization_id	UUID	Yes	Tenant-scoped

4.3 VariantAttributeValues - Storing the Actual Values

Field	Type	Required	Notes
value_id	UUID	Yes	Primary key
variant_id	FK to ProductVariants	Yes	Which variant this value belongs to
attribute_id	FK to AttributeDefinitions	Yes	Which attribute
value	NVARCHAR(500)	Yes	The actual value, stored as text regardless of data_type. Application layer converts to the correct type for validation and display. For MULTI_SELECT, stored as JSON array.
organization_id	UUID	Yes	Tenant-scoped

4.4 Worked Example: Laptop Category Attributes

Attribute	Display Name	Data Type	Required?	Searchable?	Control
cpu	CPU	DROPDOWN	Yes	Yes	DROPDOWN
ram	RAM	DROPDOWN	Yes	Yes	DROPDOWN
storage	Storage	DROPDOWN	Yes	Yes	DROPDOWN
screen_size	Screen Size	DECIMAL	No	No	NUMBERBOX
color	Color	DROPDOWN	No	Yes	DROPDOWN
gpu	GPU	TEXT	No	Yes	TEXTBOX

5. Dynamic UI Rendering

When a user selects a category on the product/variant form, the system loads that category's configured attributes from CategoryAttributes and renders the form dynamically. No category-specific hardcoded screens exist anywhere in the frontend.

5.1 Rendering Flow

- User selects Category = 'Laptop' on the product form.
- React calls GET /api/categories/{categoryId}/attributes - returns the ordered list of AttributeDefinitions linked to Laptop via CategoryAttributes.
- For each attribute, React renders the appropriate control: TEXTBOX for TEXT, NUMBERBOX for NUMBER/DECIMAL, DATEPICKER for DATE, TOGGLE for BOOLEAN, DROPDOWN for DROPDOWN (options from dropdown_options JSON), MULTI_SELECT for MULTI_SELECT.
- Required attributes show a red asterisk. Validation is enforced client-side and server-side.
- Changing the category clears the existing attribute values and loads the new category's attribute set.
- The same dynamic rendering is used on the variant edit form - attribute values are pre-populated from VariantAttributeValues.

5.2 Industry Examples - Same Engine, No Code Changes

Category	Attributes rendered dynamically
Laptop	CPU (dropdown), RAM (dropdown), Storage (dropdown), Screen Size (number), Color (dropdown), GPU (text)
Mobile Phone	Storage (dropdown), RAM (dropdown), Color (dropdown), IMEI Required (boolean), Screen Size (number)
Bag	Material (dropdown), Capacity (text), Color (dropdown), Waterproof (boolean)
Medicine	Dosage (text), Form (dropdown: Tablet/Capsule/Syrup), Active Ingredient (text), Expiry Required (boolean)
Garment	Size (dropdown: XS/S/M/L/XL/XXL), Color (dropdown), Material (dropdown: Cotton/Polyester/Silk), Gender (dropdown)
Auto Part	Make (dropdown), Model (text), Year Range (text), OEM Part No (text), Aftermarket (boolean)

6. Category Attribute Configuration (Admin)

Navigation	Inventory > Categories > {category_name} > Configure Attributes
Route	/inventory/categories/{id}/attributes
Access	Admin and Inventory Manager only
Layout	Drag-and-drop list of available attributes. Add/remove attributes from the category. Set per-category is_required override. Set display_order via drag. Preview the rendered form in a side panel.
No code changes	Adding a new attribute to a category takes effect immediately on the next product/variant form load. No deployment, no restart, no developer involvement.
Validation config	Per attribute: required toggle, regex pattern (for text fields), min/max (for number fields). All enforced both client-side and server-side.

7. Downstream Module Impact Analysis

Core change

Every FK that currently points to `product_id` in transactional line tables changes to `variant_id`. The `product_id` is still accessible via the variant's parent, but the direct FK is `variant_id`. This is the single most impactful change in this addendum and affects every module listed below.

7.1 Purchase Orders (Recommendation: Option B - Purchase Variant)

FK change	POLines. <code>product_id</code> changed to POLines. <code>variant_id</code> (FK to ProductVariants)
UI change	Product picker on PO line form: first select Product (family), then select Variant (SKU) from a sub-dropdown. For simple products (one default variant), the variant is auto-selected and the sub-dropdown is hidden.
Price population	POLine. <code>unit_price</code> defaults from ProductVariant. <code>purchase_price</code> , not Product. <code>purchase_price</code> (which no longer exists on Product)
Why Option B	A PO for 'Dell Latitude 5450' is meaningless without specifying which configuration. The supplier needs the exact SKU to fulfil. Option A (purchase product) would require post-PO disambiguation at GRN time, which creates reconciliation problems.

7.2 Goods Receipt (GRN)

FK change	GRNLines. <code>product_id</code> changed to GRNLines. <code>variant_id</code>
Auto-population	GRN lines created from a PO inherit the <code>variant_id</code> from the PO line. No manual variant selection at GRN time.
Barcode scan	Scanning a barcode at GRN receiving resolves to the specific variant (ProductVariants. <code>barcode</code>), not the product family. If the scanned variant does not match the PO line's variant, the system warns.
last_purchase_price	On GRN approval, ProductVariant. <code>last_purchase_price</code> is updated (not Product. <code>last_purchase_price</code> , which no longer exists on Product).

7.3 Inventory

FK change	InventoryItems: <code>product_id</code> changed to <code>variant_id</code> . Each variant at each warehouse = one InventoryItem row.
Tracking	<code>qty_on_hand</code> , <code>qty_reserved</code> , <code>qty_available</code> are all at the VARIANT + WAREHOUSE level. The stock of 'Dell Latitude i7/16/512' at 'Main Warehouse' is one InventoryItem row.
Product-level view	A product-level stock summary (total across all variants) is a computed view: SUM(<code>qty_on_hand</code>) WHERE <code>variant_id</code> IN (variants of <code>product_id</code>). Not stored, computed on demand.
Stock Ledger	InventoryLedgerEntries. <code>product_id</code> changed to <code>variant_id</code> . Every stock movement (GRN, issue, return, adjustment) is recorded per variant.
Batch/Serial	Batch and serial tracking operate per variant (unchanged in concept, just the FK moves from product to variant).

7.4 Material Issue Request (MIR)

FK change	MIRLine.product_id changed to MIRLine.variant_id
UI change	MIR line form: select Product, then Variant. Simple products auto-select the default variant.
Stock availability	Checked at variant + warehouse level (already the case for InventoryItems, now scoped by variant_id instead of product_id)
Reservation	StockReservation.product_id changed to variant_id. Reserved stock is per variant.
Material Issue Voucher	MIVLine.product_id changed to variant_id. Batch/serial selection operates per variant.

7.5 POS (if implemented)

Barcode scan	Resolves to variant_id via ProductVariants.barcode. Adds the specific variant to the sale.
Product search	Searching 'Dell' shows the product; selecting it shows variant picker. Barcode scan bypasses the picker and goes directly to the variant.
Price	SalesLine.unit_price sourced from ProductVariant.selling_price
Stock decrement	On sale, InventoryItem.qty_on_hand decremented for the specific variant at the POS store's warehouse.

7.6 Reports, Supplier Scorecard, Cost Allocation

All reports that currently group/filter by product_id are updated to group/filter by variant_id with a parent product_name column for readability. The Supplier Scorecard (Addendum 23) scores per-GRN which already operates at the PO/GRN level and is unaffected structurally. Project and Department cost allocation (Addendum 20) posts per MIV line, which now references variant_id - the cost amount calculation (qty x unit_cost) is unchanged, just the FK level shifts.

8. Search Strategy

Attributes marked `is_searchable = true` are indexed for product/variant search. When a user searches 'Dell i7', the system should return the Dell Latitude variant with i7 CPU.

8.1 Search Implementation

- Build a denormalised search index view or materialised table: `ProductSearchIndex` with `variant_id`, `product_name`, `sku`, `barcode`, `variant_name`, and a concatenated `searchable_text` field containing all searchable attribute values joined with spaces.
- `searchable_text` for the i7 variant example: 'Dell Latitude 5450 i7 16GB 512GB SSD Silver' (product name + variant name + all searchable attribute values).
- `GET /api/products/search?q=dell+i7` performs a CONTAINS or full-text search against `searchable_text`, returning matching variants with their parent product info.
- The search index is rebuilt (Hangfire job) whenever a variant's attributes change or a new variant is created. For performance, this is an async background rebuild, not inline on every save.
- SQL Server Full-Text Search index on `searchable_text` provides the best query performance for CONTAINS queries across large product catalogs.
- POS barcode scan: exact match on `ProductVariants.barcode` (indexed, instant lookup, no full-text search needed).

9. Migration Strategy

Simplified

Since the project is still in development, all existing product data and dependent transactional records (POS, GRNs, invoices, MIRs, inventory, ledger entries) are DROPPED and rebuilt cleanly. This eliminates the complexity of in-place data migration with FK rewiring.

9.1 Migration Steps

#	Step	Detail
1	DROP dependent transactional data	TRUNCATE or DROP all rows from: POLines, PurchaseOrders, GRNLines, GRNs, Invoices, MIRLines, MIR, MIVLines, MIV, StockReservation, InventoryItems, InventoryLedgerEntries, MasterProductLedger, MasterFinancialLedger, SupplierLedgerEntries, SupplierPayments, ProjectCostLedger, DepartmentCostLedger, Wastage, MaterialReturn, MaterialConsumption. Order respects FK dependencies (children before parents).
2	DROP existing Products	TRUNCATE Products, Categories (optional - may keep categories), SubCategories (optional).
3	Add new tables	EF Core migrations: AddProductVariants, AddAttributeDefinitions, AddCategoryAttributes, AddVariantAttributeValues, AddProductSearchIndex
4	Modify existing Product table	Remove: purchase_price, selling_price, barcode, last_purchase_price columns. Keep all other columns.
5	Change FK references across all modules	POLines, GRNLines, InventoryItems, MIRLines, MIVLines, StockReservation, InventoryLedgerEntries, MasterProductLedger: rename product_id to variant_id, change FK target from Products to ProductVariants.
6	Seed sample data	Create sample categories with attributes configured, sample products with variants, for development/testing.
7	Rebuild all module services	Update every service/repository that reads or writes product_id to read/write variant_id instead. Update DTOs, AutoMapper profiles, FluentValidation rules.
8	Update React frontend	Product picker component: two-level (Product then Variant). Dynamic attribute form renderer. Variant management screen on Product detail.

9.2 Rollback Strategy

Since this is a development environment with no production data, rollback is straightforward: revert the EF Core migrations (dotnet ef database update {previous-migration}), restore the original Product table columns, and re-seed sample data from the pre-variant schema. A Git branch protects the codebase; database rollback is migration-based.

10. API Endpoints

Method	Endpoint	Description
GET	/api/products	Product list (family level) with variant count per product
GET	/api/products/{id}	Product detail with all variants and their attribute values
POST	/api/products	Create product + at least one default variant
PUT	/api/products/{id}	Update product family fields
GET	/api/products/{id}/variants	List variants for a product
POST	/api/products/{id}/variants	Create a new variant for a product
PUT	/api/variants/{variantId}	Update variant fields (SKU, prices, barcode, attributes)
DELETE	/api/variants/{variantId}	Deactivate variant (never hard delete if transacted)
GET	/api/categories/{categoryId}/attributes	Attributes configured for this category (drives dynamic form)
POST	/api/categories/{categoryId}/attributes	Link an attribute to a category with display order and required override
DELETE	/api/categories/{categoryId}/attributes/{attrId}	Remove attribute from category
GET	/api/attributes	List all AttributeDefinitions for the tenant
POST	/api/attributes	Create a new attribute definition
PUT	/api/attributes/{id}	Update attribute (name, type, options, searchable, etc.)
GET	/api/products/search?q={query}	Full-text search across product names, variant names, SKUs, barcodes, and searchable attribute values
GET	/api/variants/lookup?barcode={code}	Exact barcode lookup for POS/GRN scanning

11. Performance and Indexing

Index / Strategy	Purpose
IX_ProductVariants_SKU (UQ, filtered by org id)	Fast SKU lookup, enforces uniqueness per tenant
IX_ProductVariants_Barcode (UQ nullable, filtered by org id)	Instant barcode scan resolution for POS and GRN
IX_ProductVariants_ProductId	Fast variant listing for a product
IX_VariantAttributeValues_VariantId	Fast attribute loading for a variant
IX_CategoryAttributes_CategoryId	Fast attribute-set loading for dynamic form rendering
FT_ProductSearchIndex_SearchableText	SQL Server Full-Text index for CONTAINS queries on product search
IX_InventoryItems_VariantId_WarehouseId (UQ)	One stock row per variant per warehouse - critical for stock lookups
Caching: CategoryAttributes per category	IMemoryCache with 10-min TTL. Category attribute sets change rarely - cache avoids per-form-load DB hit.
Caching: AttributeDefinitions per tenant	IMemoryCache with 10-min TTL. Attribute catalog changes rarely.

12. Role-Based Access

Action	Admin	Inv. Mgr	Proc. Mgr	Others
Create/edit Products and Variants	Full	Full	None	None
Configure Category Attributes	Full	Full	None	None
Create/edit Attribute Definitions	Full	Full	None	None
View products and variants	Full	Full	Full	All roles (read)
Select variant on PO/MIR lines	Full	Full	Full	Purchase Off., Requester
Barcode scan (POS/GRN)	Full	Full	Full	Warehouse Op., Cashier

- End of Addendum -